Spark Performance

Techniques for Improving Spark Performance

- Tuning Spark configuration to improve scaling for large workloads, enhance parallelism, and minimize memory starvation among Spark executors.
- Using caching and persistence with appropriate levels to expedite access to frequently used data sets.
- Using the Spark web UI to gain visual perspective on performance.
- Using the right data format.
- Reduce the number of passes by using chunkier lambdas.

Viewing and Setting Spark Configuration

- There are three ways to get and set Spark configuration properties.
  - By specifying them in the configuration files found in `$SPARK_HOME/conf` folder.
  - By specifying directly on the `spark-submit` command as a command-line argument.

    ```
    spark-submit --conf "spark.executor.memory=2g"
        BetterInvertedIndex.py --master local[*]
    ```

  - By specifying them inside our application.

    ```
    spark.conf.get("spark.sql.shuffle.partitions")
    '200'
    spark.conf.set("spark.sql.shuffle.partitions", 5)
    spark.conf.get("spark.sql.shuffle.partitions")
    '5'
    ```

- **Order of precedence**: From least to highest. (1) Configuration files, (2) command line arguments to spark-submit (3) Set on `SparkSession` configuration in the Spark application.

Scaling Spark for Large Workloads

- **Static versus dynamic resource allocation**: In static allocation (the default), we set a fixed number of executors and Spark queues up jobs if all executors are busy. Dynamic allocation allows for varying number of executors. It is useful for streaming or in multi-tenant environments.

```
spark.dynamicAllocation.enabled true
spark.dynamicAllocation.minExecutors 2
spark.dynamicAllocation.schedulerBacklogTimeout 1m
spark.dynamicAllocation.maxExecutors 20
spark.dynamicAllocation.executorIdleTimeout 2m
spark.shuffle.service.enabled true
```

Scaling Spark for Large Workloads (2)

- **Configuring executors' memory and the shuffle service**. The amount of memory available is controlled by `spark.exceutor.memory`. It is divided into execution memory (60%), storage memory (40%), and reserved memory (300MB).
- Execution memory is used for shuffles, joins, sorts, and aggregations. Storage memory is used for user data structures and partitions
- During map and shuffle, there is heavy I/O activity. This can result in a bottleneck because the default configurations are suboptimal for large scale Spark jobs.

```
spark.driver.memory  1g
spark.shuffle.file.buffer 32k (recommended 1MB)
spark.file.transferTo true  (set it to false to force use of
    buffer)
spark.shuffle.unsafe.file.output.buffer 32k (use 1m for
   larger, used for merges in shuffle)
spark.io.compression.lz4.blockSize  32k (increase to 512k)
spark.shuffle.service.index.cache.size 100m
```

- **Maximize Parallelism**. Use repartition() method to rebalance the workload across cores and threads.

Scaling Spark for Large Workloads (3)

```
ds = spark.read.text("../README.md").repartition(16)
print(ds.rdd.getNumPartitions())  # 16

num_df = spark.range(1000 * 1000 * 1000).repartition
    (16)
print(num_df.rdd.getNumPartitions())  # 16
```

Maximizing Spark Parallelism

The 1:1:1 Execution Mapping

- **Atomic Unit of Parallelism:** 1 Partition $\rightarrow$ 1 Task $\rightarrow$ 1 Dedicated Thread.
- **Physical Limitation:** A single CPU core can process only **one partition at a time**.
- **Data Structure:** Partitions map directly to physical chunks or blocks of contiguous data on disk.

The Core-to-Partition Formulas

To prevent resource starvation, you must scale partitions relative to total cluster cores:

Resource Under-utilization (Bad)

$$\text{Partitions} < \text{Total Executor Cores}$$

Result: Idle CPU cores, wasted cluster capacity, and unallocated threads.

Maximum Hardware Saturation (Ideal)

$$\text{Partitions} \geq \text{Total Executor Cores}$$

Result: 100% core utilization. Over-provisioning partitions keeps all cores busy as fast tasks finish.

Configuration Properties: Core Parallelism

You must configure Spark explicitly to enforce this ideal core-to-partition ratio.

- `spark.default.parallelism`
  - Controls the number of partitions in RDDs returned by transformations like `join` or `groupByKey` when no partition count is passed.
  - *Recommendation:* Set this to $2\times$ to $4\times$ the total number of cores in your cluster.
- `spark.executor.cores`
  - Defines the number of concurrent tasks an individual executor can run.
  - *Best Practice:* Typically set to 4 or 5 per executor to optimize I/O throughput without choking on HDFS/storage locks.

Configuration Properties: SQL & Shuffle Partitions

When moving from raw file reads to structured queries, operations that cause a shuffle require distinct tuning parameters.

- `spark.sql.shuffle.partitions`
  - Controls the number of partitions used when shuffling data for Joins or Aggregations in DataFrames/Spark SQL.
  - *Default Warning:* Defaults to **200**, which is often too high for tiny datasets (causing small file overhead) and too low for terabyte-scale jobs (causing Out-Of-Memory errors).
- `spark.sql.adaptive.enabled`
  - Enables Adaptive Query Execution (AQE).
  - Automatically coalesces post-shuffle partitions at runtime based on size, safely mitigating poorly guessed static configurations.

Caching and Persistence of Data

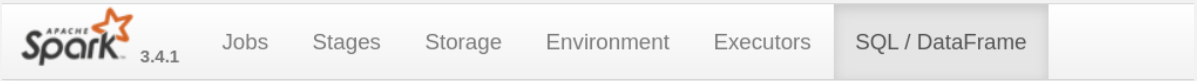
- Use the `cache()` call on a DataFrame or RDD to cache it. The caching only happens after we take an action to realize the data frame or RDD.
- Use the `persist()` method for more control how the data is cached.

`MEMORY_ONLY`	Objects in memory
`MEMORY_ONLY_SER`	Serialized in memory
`MEMORY_AND_DISK`	Objects in memory, serialized on disk
`DISK_ONLY`	serialized on disk
`OFF_HEAP`	Stored off-heap.
`MEMORY_AND_DISK_SER`	serialized in memory and on disk

- **When to cache?** A DataFrame used repeatedly in iterative machine learning training, during frequent transformations during ETL or building data pipelines.
- **When not to cache?** DataFrame is too big to fit in memory. Inexpensive transformation on a DataFrame not requiring frequent use (regardless of size).
- Spark caching example

SparkUI

- Spark offers an elaborate web UI that allows to examine various components of our applications.



- Debugging through the UI is more like being a detective, following trails of bread crumbs.

Using Chunkier Lambdas to Improve Performance (1)

- The idea is to process data in batches or entire partitions rather than one record at a time
- Standard transformations like `map` and `filter` invoke your lambda function once for every single row. Switching to `mapPartitions` passes an iterator of an entire partition to your function at once.
- Group consecutive map and filter steps into a single mapPartitions lambda to avoid creating intermediate iterators and extra execution steps
- Use vectorized operations with libraries like NumPy or Pandas (backed by Arrow) to process entire batches of data at once.
- Tune batch sizes. Adjust `spark.sql.execution.arrow.maxRecordsPerBatch` (default is 10,000) to balance memory usage against serialization throughput.

Using Chunkier Lambdas to Improve Performance (2)

- Aggregate within the partition first. Use `mapPartitions` to compute partial sums, counts, or deduplications in a local dictionary before emitting records.
- Memory pressure. Chunking holds multiple records in executor memory simultaneously. Keep batch sizes bounded so large partitions do not spill to disk or crash executors.
- Bypassing Catalyst. Custom lambdas in mapPartitions are opaque to Spark's Catalyst optimizer. Always prefer native Spark SQL functions first when the logic does not require custom code or external libraries.